

LITHO Mainnet Exchange API Reference

Last reviewed: 2026-07-29

This document provides exchange-oriented examples for native LITHO and LEP100 assets on EVM chain 9005. Replace placeholders and use a private exchange node for production. Public endpoints are intended for integration and independent comparison, not as the sole custody backend.

1. Endpoints

Interface	Public endpoint
EVM JSON-RPC	POST https://rpc-mainnet.litho.ai
EVM WebSocket	wss://rpc-mainnet.litho.ai/websocket
Cosmos REST	https://api-mainnet.litho.ai
gRPC TLS	grpc-mainnet.litho.ai:9090
Read-only CometBFT	GET https://rpc-mainnet.litho.ai/{allowlisted_route}

The public EVM proxy is limited to 25 requests/second/IP with a bounded burst; REST is limited to 30 requests/second/IP. A single eth_getLogs query is also bounded by node-side block-range and result caps. Paginate all scans.

2. JSON-RPC request helper

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc":"2.0","id":1,"method":"METHOD","params":[]}'
```

Every client must reject a response containing error, an unexpected id, or an invalid jsonrpc version. Quantities are hexadecimal integers, not decimal strings.

3. Network and node health

EVM chain ID

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc":"2.0","id":1,"method":"eth_chainId","params":[]}'
```

Expected result: 0x232d.

Best EVM height

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc":"2.0","id":1,"method":"eth_blockNumber","params":[]}'
```

Convert the hex result to an integer.

Cosmos identity and height

```
curl -fsS https://rpc-mainnet.litho.ai/status | jq '{
  chain_id: .result.node_info.network,
  height: .result.sync_info.latest_block_height,
  block_hash: .result.sync_info.latest_block_hash,
  block_time: .result.sync_info.latest_block_time,
  catching_up: .result.sync_info.catching_up
}'
```

Require chain ID `lithosphere_9005-1`, current time, advancing height, and `catching_up=false`.

Peer and validator state

```
curl -fsS https://rpc-mainnet.litho.ai/net_info | jq '.result.n_peers'
curl -fsS https://rpc-mainnet.litho.ai/validators | jq '.result'
```

The validator response contained one validator at review time. Alert on any unexpected validator-set or voting-power change.

4. Address and account validation

EVM syntax

Use a maintained Ethereum address library. A valid address is 20 bytes, normally rendered as `0x` plus 40 hex characters. Require a valid EIP-55 checksum when mixed case is supplied. Reject `0x0000000000000000000000000000000000000000000000000000000000000000`.

Determine whether an EVM address has contract code

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc":"2.0","id":1,"method":"eth_getCode","params":["0xADDRESS","latest"]}'
```

`0x` normally means an externally owned account at the selected block. A non-empty result means contract code is present. Contract accounts are valid on chain, but an exchange may reject them as deposit destinations according to its withdrawal policy.

Cosmos account

```
curl -fsS \
https://api-mainnet.litho.ai/cosmos/auth/v1beta1/accounts/litho1ADDRESS \
| jq '.account'
```

An HTTP not-found response does not necessarily make an address invalid: a syntactically valid account may not yet have on-chain state. Validate Bech32 locally first.

5. Balances and nonces

Native EVM balance

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc":"2.0","id":1,"method":"eth_getBalance","params":["0xADDRESS","latest"]}'
```

Result is an integer number of `ulitho` encoded as hex.

EVM transaction count / nonce

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc": "2.0", "id": 1, "method": "eth_getTransactionCount", "params": ["0xADDRESS", "pending"]}'
```

Use `pending` for constructing a new withdrawal and `latest` for confirmed state. The exchange must serialize withdrawals per sender or use a durable nonce allocator.

Cosmos native balance

```
curl -fsS \  
'https://api-mainnet.litho.ai/cosmos/bank/v1beta1/balances/litho1ADDRESS/by_denom?denom=ulitho' \  
| jq '.balance'
```

Amount is a decimal-string integer in `ulitho`.

LEP100 balance

The `balanceOf(address)` selector is `0x70a08231`. Append the address left-padded to 32 bytes:

[illegible]

Use an ABI library rather than hand-building calldata in production.

6. Blocks and transaction details

Load block with full transactions

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc":"2.0","id":1,"method":"eth_getBlockByNumber","params":["0xBLOCK_NUMBER",true]}'
```

Store block number, block hash, parent hash, timestamp, and transaction hashes. Before credit, reload the block by number and confirm its hash has not changed.

Transaction object

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc":"2.0","id":1,"method":"eth_getTransactionByHash","params":["0xTX_HASH"]}'
```

Transaction receipt

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc":"2.0","id":1,"method":"eth_getTransactionReceipt","params":["0xTX_HASH"]}'
```

A pending/unknown transaction returns `null`. A mined transaction must have a receipt. Require `status == 0x1`; `0x0` means EVM execution failed. Also verify `transactionHash`, `blockHash`, `blockNumber`, `to`, and all relevant logs.

Cosmos transaction by hash

```
curl -fsS \
https://api-mainnet.litho.ai/cosmos/tx/v1beta1/txs/TX_HASH \
| jq '.tx_response'
```

Require `code == 0`. Do not credit a Cosmos transfer based only on raw message contents when execution failed.

7. Account history and deposit indexing

Ethereum JSON-RPC does not define a complete transactions-by-address endpoint. The production exchange indexer should:

1. persist its last fully processed height;
2. read each canonical block by height;
3. store native transactions and receipts relevant to exchange addresses;
4. query allowlisted token logs in bounded ranges;
5. wait the approved confirmation buffer;
6. credit with an idempotency key; and
7. replay safely from an earlier checkpoint after recovery.

Do not use explorer search results as an accounting source.

8. LEP100 Transfer log scanning

Event signature:

```
Transfer(address,address,uint256)
topic0 = 0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef
topic1 = from address, left-padded to 32 bytes
topic2 = to address, left-padded to 32 bytes
data    = amount as uint256 base units
```

Filter one exact token and recipient:

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "eth_getLogs",
  "params": [{
    "fromBlock": "0xSTART",
    "toBlock": "0xEND",
    "address": "0xALLOWLISTED_TOKEN",
    "topics": [
      "0xddf252ad1be2c89b69c2b068fc378daa952ba7f163c4a11628f55a4df523b3ef",
      null,
      "0x000000000000000000000000DEPOSIT_ADDRESS_WITHOUT_0X"
    ]
  }]
}'
```

Credit by (chain_id, transaction_hash, log_index). Never infer a token from symbol alone; symbols and names are not unique.

9. LEP100 metadata calls

Function	Selector	Expected ABI result
name()	0x06fdde03	string
symbol()	0x95d89b41	string
decimals()	0x313ce567	uint8
totalSupply()	0x18160ddd	uint256

Example:

```
curl -fsS https://rpc-mainnet.litho.ai \
-H 'content-type: application/json' \
--data '{"jsonrpc": "2.0", "id": 1, "method": "eth_call", "params": [{"to": "0xTOKEN_CONTRACT", "data": "0x313ce567"}, {"latest"}]}'
```

Decode ABI output using ethers/viem/web3. Verify metadata once at listing and monitor contract code hash afterward.

10. Gas and fee queries

```
# Legacy-compatible gas price
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_gasPrice","params":[]}'

# Estimate gas for an exact transaction object
curl -fsS https://rpc-mainnet.litho.ai \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_estimateGas","params":[{"from":"0xFROM","to":"0xTO","value":"0xVALUE"}]}'
```

Do not hard-code the observed gas price. Estimate against current state, apply an exchange policy margin, and enforce a maximum fee.

11. Offline EVM signing and online broadcast

The online system may prepare an unsigned EIP-1559 transaction:

```
{
  "type": 2,
  "chainId": 9005,
  "nonce": 12,
  "to": "0xRECIPIENT",
  "value": "10000000000000000",
  "gasLimit": "21000",
  "maxFeePerGas": "200000000",
  "maxPriorityFeePerGas": "0",
  "data": "0x"
}
```

The offline/HSM zone must independently verify every field. A simplified ethers v6 illustration is:

```
import { Wallet } from "ethers";
import unsigned from "../approved-unsigned-transaction.json" with { type: "json" };

// Illustration only. Production exchanges should use an HSM/MPC signer and
// must not load raw private keys from environment variables.
const signer = new Wallet(process.env.OFFLINE_DEMO_PRIVATE_KEY);
if (Number(unsigned.chainId) !== 9005) throw new Error("wrong chain ID");
const signedRawTransaction = await signer.signTransaction(unsigned);
console.log(signedRawTransaction);
```

Broadcast signed bytes from the online node:

```
curl -fsS http://127.0.0.1:8545 \
  -H 'content-type: application/json' \
  --data '{"jsonrpc":"2.0","id":1,"method":"eth_sendRawTransaction","params":["0xSIGNED_RAW_TRANSACTION"]}'
```

Recompute the transaction hash locally, compare it with the RPC response, and then monitor the receipt. Never reassign a nonce without reconciling pending and confirmed transactions.

12. Offline Cosmos signing alternative

EVM is the recommended exchange integration. If Cosmos [litho1](#) withdrawals are supported, pin chain ID, account number, sequence, fee, memo, and denom:

```
# Online/watch-only construction
lithod tx bank send litho1FROM litho1TO 1000000000000000000ulitho \
  --chain-id lithosphere_9005-1 \
  --account-number ACCOUNT_NUMBER \
  --sequence SEQUENCE \
  --fees 1000ulitho \
  --memo 'EXCHANGE_WITHDRAWAL_ID' \
  --generate-only > unsigned.json

# Offline signer
lithod tx sign unsigned.json \
  --offline \
  --from CUSTODY_KEY \
  --chain-id lithosphere_9005-1 \
  --account-number ACCOUNT_NUMBER \
  --sequence SEQUENCE \
  --output-document signed.json

# Online broadcast through the exchange node
lithod tx broadcast signed.json \
  --node tcp://127.0.0.1:26657 \
  --broadcast-mode sync
```

Query the transaction until included and require `code == 0`.

13. Deposit-credit algorithm

For each candidate deposit:

1. Ensure local and independent nodes agree on both chain IDs and are advancing.
2. Require a canonical block and successful execution result.
3. Match the exact assigned recipient.
4. Match native LITHO or the exact allowlisted LEP100 contract.
5. Decode integer base units and enforce configured decimals.
6. Apply sanctions/risk and minimum-deposit policies.
7. Wait the approved confirmation buffer.
8. Recheck the block hash before credit.
9. Credit exactly once using a durable idempotency key.
10. Retain raw block, transaction, receipt/log, and parsing evidence.

Pause crediting on stale blocks, conflicting hashes, parser errors, token-code changes, or an unapproved validator-set/binary upgrade.

14. Useful upstream references

- Ethereum JSON-RPC: <https://ethereum.org/developers/docs/apis/json-rpc/>
- ethers v6 provider and signing APIs: <https://docs.ethers.org/v6/>
- Cosmos SDK transactions: <https://docs.cosmos.network/sdk/v0.50/learn/advanced/transactions>
- CometBFT RPC: <https://docs.cometbft.com/v0.38/spec/rpc/>